知识点总结

2025年12月23日 21:06

1. 窗口函数的标准格式

```sql
函数名() OVER (
 PARTITION BY 分组列 -- 按指定字段分组
 ORDER BY 排序列 [ASC/DESC] -- 组内排序规则
) AS 别名 -- 为结果命名
```

rank排名时候：RANK( ) OVER( ) 接上窗口函数。

- **核心要求**：
- PARTITION BY和 ORDER BY必须包裹在 OVER()中。
- 函数名（如 RANK()）后必须紧跟 OVER()。

一般UPDATE语句的逻辑执行顺序：

1. **处理FROM/JOIN子句**（如果涉及连接）：确定数据源，执行表连接操作。
2. **处理子查询**（如果存在）：优先执行子查询，获取中间结果。
3. **应用WHERE条件**：对数据进行过滤，只保留满足条件的行。
4. **执行更新操作**：对过滤后的每一行，更新指定列的值。

★ PARTITION BY 和ORDER BY 、GROUP BY的区别：
PARTITION BY 为不减少行数的分组，ORDER BY 为排序，GROUP BY为减少行数的压缩分组

1. GROUP BY（分组聚合）

- **核心**：**折叠数据**。将具有相同分组键值的行合并为一行，通常与聚合函数（SUM, AVG, COUNT, MAX, MIN）一起使用，以生成摘要信息。
- **输出变化**：查询结果的行数**减少**，通常等于唯一分组键的数量。
- **SELECT 限制**：SELECT子句中只能包含分组键和聚合函数表达式。
- **示例**：

```sql
-- 计算每个部门的总薪资（结果行数 = 部门数量）
SELECT department_id, SUM(salary) AS total_salary
FROM employees
GROUP BY department_id;
```

结果：

department_id	total_salary
10	50000
20	75000

2. PARTITION BY（创建窗口分区）

- **核心**：**定义计算范围，但不折叠。**它将数据集划分为多个“分区”或“窗口”，窗口函数在每个分区内独立进行计算，但**原始每一行都保留在结果中。**
- **输出变化**：查询结果的行数**不变。**
- **使用位置**：**必须**与窗口函数一起，放在 OVER()子句中。
- **示例**：

```sql
-- 计算每个部门内员工的薪资排名（结果行数 = 员工总数）
SELECT
    employee_id,
    department_id,
    salary,
    RANK() OVER (PARTITION BY department_id ORDER BY salary DESC) AS dept_rank
FROM employees;
```

结果：

employee_id	department_id	salary	dept_rank
1	10	30000	1
2	10	20000	2
3	20	40000	1
4	20	35000	2

3. ORDER BY（排序）

它在两个不同上下文中起作用，含义不同：

- **在主查询中**：对**最终结果集**进行全局排序。它总是最后执行。

```sql
SELECT department_id, SUM(salary) AS total_salary
FROM employees
GROUP BY department_id
ORDER BY total_salary DESC; -- 对汇总后的结果进行排序
```

- **在 OVER()子句中**：与 PARTITION BY配合使用，**定义窗口函数在分区内的计算顺序。**这对排名函数（RANK, ROW_NUMBER）、累计函数（SUM）等至关重要。

```sql
-- 计算每个部门内薪资的累计和（按薪资升序累计）
SELECT
    employee_id,
    department_id,
    salary,
    SUM(salary) OVER (
        PARTITION BY department_id
        ORDER BY salary ASC -- 此ORDER BY决定了累计的顺序
    ) AS running_total
FROM employees;
```

特性	GROUP BY	PARTITION BY
主要目的	**分组聚合**，将数据压缩为汇总行。	**分组计算**，定义窗口函数的计算范围，**不减少行数**。
行数影响	减少行数（每组返回一行）。	**不减少**行数，保持原表行数。
能否与聚合函数同用	**是**，标准用法（如SUM, COUNT）。	**是**，但必须与窗口函数（OVER()）结合使用。
典型使用场景	生成汇总报告，如"每个部门的总薪资"。	计算相对于分组的值，如"每个部门内员工的薪资排名"。

ORDER BY(在 OVER()中)	ORDER BY(在主查询中)
在**每个分区内**为窗口函数**定义计算顺序**。	对查询**最终结果集**进行排序。
不影响行数，仅影响窗口计算逻辑。	不影响行数，仅改变行的顺序。
必须与窗口函数结合使用。	用于排序最终结果，与聚合函数无关。
在窗口函数中定义顺序，用于计算排名、累计和等。	对最终输出的结果进行升序或降序排列。

关键对比与总结

对比维度	GROUP BY	PARTITION BY	ORDER BY(在OVER()中)
哲学	"折叠并摘要"	"分区并计算，保留细节"	"定义计算顺序"
结果行数	减少（每组一行）	**不变**（每行保留）	不变
与聚合函数	必须一起使用，进行**最终聚合**	可选，进行**窗口内聚合**（如累计）	决定窗口聚合的顺序
常见搭配	SUM(), COUNT(), AVG()等	ROW_NUMBER(), RANK(), SUM() OVER()等	在窗口函数中必须，用于排名、累计

简单记忆:

- 想要**汇总统计**（如总计、平均） -> 用 **GROUP BY**。
- 想要在**保留所有行的同时**进行分组计算（如排名、累计） -> 用 **PARTITION BY**。
- 想要**对最终结果排序** -> 在主查询末尾用 **ORDER BY**。
- 想要**控制窗口函数的计算顺序**（如按时间累计） -> 在 OVER()子句中用 **ORDER BY**。

2. **对窗口函数结果的影响**

场景	示例（RANK()）	效果
仅用 PARTITION BY	`RANK() OVER (PARTITION BY department ORDER BY NULL)`	每组内所有行排名相同（无序状态）。
仅用 ORDER BY	`RANK() OVER (ORDER BY salary DESC)`	全局按工资排名，无分组。
组合使用	`RANK() OVER (PARTITION BY department ORDER BY salary DESC)`	**每组内**按工资从高到低排名。

4.12 外连接查询

外连接查询分为左外连接查询和右外连接查询

语法

```sql
--左外连接
select 字段1, 字段2..
from 表1 left outer join 表2 on 过滤条件;

--右外连接
select 字段1, 字段2..
from 表1 right outer join 表2 on 过滤条件;
```

> 左外连接和右外连接有一点区别：
> 左外连接：是表1和表2的交集再并上表1的其他数据
> 右外连接：是表1和表2的交集再并上表2的其他数据

4.13 子查询

> 当我们进行语句查询的时候，总会遇到我们需要的条件需要通过另一个查询语句查询出来后才能进行，就是说A 查询语句需要依赖B 查询语句的查询结果，B 查询就是子查询，A 查询语句就是主查询，一个SQL语句可以包含多个子查询。

语法

```sql
select username
from user
where age =(
    select avg(age)
    from userInfo
)
```

简单来说，**非标准函数**指的是某个数据库系统**自己特有的函数**，而不是由SQL**标准**所定义的、被各大数据库厂商普遍支持的函数。

你可以把它理解为数据库领域的“方言”。

核心区别

特性	标准函数 (SQL Standard)	非标准函数 (Vendor-specific)
定义	由国际组织（如ISO）制定的SQL标准中明确定义的函数。	由某个数据库厂商（如MySQL、Oracle、SQL Server）自行定义和实现的函数。
通用性	**高**。在大多数主流数据库（如MySQL、PostgreSQL、Oracle、SQL Server）中都存在，且名称、用法基本相同。	**低**。通常只在**特定**的数据库系统中可用。
目的	为了保证代码的可移植性。用标准函数写的SQL语句，更容易从一个数据库迁移到另一个数据库。	为了解决特定数据库的独特需求，或提供标准中未包含的、更便利或更高效的解决方案。
举例	`COUNT()`, `SUM()`, `UPPER()`, `LOWER()`, `ROUND()`, `CURRENT_DATE`, `PERCENT_RANK()`	MySQL的 `IFNULL()`在其他数据库中可能是 `ISNULL()`或 `NVL()`。

★ 子查询中不能缺少别名，派生表必须具有别名，否则会报错

内连接、自连接、外连接（左、右、全）、交叉连接 。

内连接（inner join/join）：只有两个元素表相匹配的才能在结果集中显示。

外连接outer join：可进一步分为左外连接left outer join和右外连接right outer join，（简称左连接left join，右连接 right join）。

左外连接:左边为驱动表，驱动表的数据全部显示，匹配表的不匹配的不会显示。

右外连接:右边为驱动表，驱动表的数据全部显示，匹配表的不匹配的不会显示。

全外连接（full join/full outer join）：连接的表中不匹配的数据全部会显示出来。

交叉连接（cross join）： 笛卡尔效应，即不加任何条件，达到 M*N 的结果集。

在结构化查询语言（SQL）中，功能主要分为以下几类：

- DDL（数据定义语言）：用于定义或修改数据库结构，如 `CREATE`、`ALTER`、`DROP`。
- DML（数据操纵语言）：用于操作数据，如 `SELECT`、`INSERT`、`UPDATE`、`DELETE`。
- DCL（数据控制语言）：用于权限管理，控制用户对数据库的访问。
- **数据库事务**：通过 `COMMIT`、`ROLLBACK`等语句实现事务控制。

题目中的 `GRANT`（授权）和 `REVOKE`（撤销权限）是专门用于管理数据库用户权限的语句，因此属于 DCL 功能。

Step 2：**回顾 MySQL 的 `LEFT()`函数语法**

```sql
LEFT(str, length)
```

- **作用**：从字符串 `str`的左侧开始截取 `length`个字符。
- **关键点**：仅需两个参数（字符串 + 截取长度），无第三个参数。

`group by app_id, version_code`

这行代码 `group by app_id, version_code`是 SQL（**结构化查询语言）中的一个子句，用于对查询结果进行分组。**

它的具体含义和作用如下：

1. **核心动作：分组**
   - 数据库会将查询到的数据行，按照 `app_id`（应用ID）和 `version_code`（版本代码）这两个字段**值的组合**进行归类。
   - 所有 `app_id`和 `version_code`都相同的行会被分到同一个组里。
2. **分组逻辑**：
   - 首先按 `app_id`分组。
   - 在同一个 `app_id`内，再按 `version_code`进行细分。
   - 例如，数据中有 `(app_1, v1)`, `(app_1, v2)`, `(app_2, v1)`，那么会分成三个组：`app_1`下的v1组、`app_1`下的v2组、`app_2`下的v1组。
3. **主要目的**：
   - 它通常与 **聚合函数**（如 `COUNT()`, `SUM()`, `AVG()`, `MAX()`）一起使用，目的是为了**对每个组进行统计计算**，而不是查看每一条原始记录。
   - 例如，结合 `SELECT app_id, version_code, SUM(download_count)`，就可以计算出**每个应用的每个版本的总下载量。**

简单来说，这行代码的意思是：

“请把数据按照‘应用ID’和‘版本号’这两个维度结合起来分类，这样我就可以对每一个‘应用-版本’组合进行单独的统计分析了。”

★ 在子表元素大于1时， 不能用= 要用in in（子表）。
即多个返回查询值不能用=，要用in 。

★ 在sql 中，只要有null，参与计算结果就是null。但是对于分组函数来说，会自动排除null，统计其他非NULL的值。
计算中（如sum）：Null加任何数都为null。
分组函数（如count）：会自动排除null，统计其他非NULL的值。

INNER JOIN 只返回两个表中连接列匹配的行。当连接列有NULL值时，NULL不会匹配任何值（因为NULL不等于任何值，包括它本身

在 SQL 中，`COUNT(1)`用于统计查询结果集的行数。

- **1是常量**：这里的 1表示一个非 NULL 的固定值，每一行都会返回 1，然后 `COUNT`函数对所有非 NULL 值计数。
- **与 `COUNT(*)`等价**：在主流数据库（如 MySQL、PostgreSQL、SQL Server 等）中，`COUNT(1)`和 `COUNT(*)`的执行效果相同，都会统计所有行（包括 NULL 值行）。优化器通常会对这两种写法做相同处理。
- **区别于 `COUNT(列名)`**：`COUNT(列名)`会忽略该列为 NULL 的行，而 `COUNT(1)`不会忽略任何行。

示例：

```sql
SELECT COUNT(1) FROM users;  -- 返回 users 表的总行数
```

注意：`COUNT(1)`是 ANSI SQL 标准写法之一，可跨数据库使用，但更常见的写法是 `COUNT(*)`，两者性能无差异。

★ COUNT(1) 与 COUNT(*)等价，都会统计所有行（包括 NULL 值行）。

3. 核心知识点总结

通过这道题，我们可以清晰地区分几个关键概念：

1. `WHEREVS HAVING`：
   - `WHERE`：在分组聚合**之前**过滤**行**。用于指定哪些**原始记录**参与计算。
   - `HAVING`：在分组聚合**之后**过滤**组**。用于指定哪些**分组结果**符合条件（条件通常包含聚合函数，如 `HAVING COUNT(*) > 5`）。
2. **聚合函数的正确使用**：
   - `COUNT(*)`或 `COUNT(1)`：用于统计**行数**。
   - `SUM(column_name)`：用于对某个数值列**求和**。
3. SQL **子句的顺序**：
   关键字的书写顺序是固定的，不能随意调换。常见的核心顺序是：

```sql
SELECT ...
FROM ...
WHERE ...        -- 先过滤行
GROUP BY ...     -- 再分组
HAVING ...       -- 最后过滤组
ORDER BY ...
```

在 SQL 中，`DEFAULT` 是一个关键字，用于为数据库表中的某个字段（列）定义一个**默认值**。

它的核心作用是：当向表中插入新记录时，**如果插入语句没有为该字段指定具体的值，数据库系统将自动使用这个预先定义的默认值来填充该字段。**

主要用途和功能：

1. **确保数据完整性**：防止因遗漏值而出现意外的 `NULL`或不一致的数据，为字段提供一个合理的、有意义的"后备"值。
2. **简化插入操作**：在编写 `INSERT`语句时，可以省略那些有合理默认值的字段，使代码更简洁。
3. **提供初始状态**：对于记录状态的字段（如 `status`）、时间戳字段（如 `create_time`）等，定义默认值可以自动为记录初始化。

基本语法：

通常在创建表时定义：

```sql
CREATE TABLE 表名 (
    字段名 数据类型 DEFAULT '默认值'
);
```

在MySQL中，DECIMAL(19,4)表示 **总长度19位，其中小数部分占4位。**对于插入的整数（如50034），实际存储为50034.0000，但查询时会省略末尾的零，直接显示为整数50034。以下是具体分析：

★ 联合索引遵循最左前缀原则，即查询从索引的最左边列开始且连续使用索引列时索引有效。表有联合索引 a,b,c。

A （a）选项 a 是最左列，索引有效；C (ac)选项 a,c 中 a 是最左列且 c 在联合索引中，也是连续使用索引列，索引有效；B（b） 选项 b 不是最左列，D（bc） 选项 b,c 起始不是最左列，索引均无效。

`ROW_NUMBER()`是 SQL 中功能强大的**窗口函数**，用于为结果集中的每一行分配一个**唯一的、连续的整数序号**（从1开始）。

1. 核心语法

```sql
ROW_NUMBER() OVER (
    [PARTITION BY column1, column2, ...]
    ORDER BY column3 [ASC|DESC], ...
) AS rank_column
```

- `OVER()`**子句**：定义窗口函数的作用范围。
- `PARTITION BY`（可选）：
  - 将结果集划分为多个"分区"或"组"。
  - 在每个分区内，`ROW_NUMBER()`会**独立重置并从1开始重新编号。**
  - 如果省略，则将整个结果集视为一个分区。

- ORDER BY（必需）：
  - 决定**分区内**的排序规则，序号将严格按照此顺序生成。
  - 这是必须指定的子句，它决定了编号的顺序逻辑。

2. **核心特点**

- **唯一性**：即使两行的排序字段值完全相同，ROW_NUMBER()也会分配**不同的序号**（例如：1, 2）。这一点与 RANK()和 DENSE_RANK()不同。
- **连续性**：序号是连续的，不会跳过任何数字。
- **确定性**：在相同的 PARTITION BY和 ORDER BY下，每次执行返回的序号是一致的。

特性	视图（View）
本质	虚拟表，由查询定义
用途	简化复杂查询、数据安全、逻辑抽象
创建方式	CREATE VIEW ... AS SELECT ...
返回形式	结果集（多行多列）

步骤1：明确SUM函数对NULL的处理规则

在MySQL中，**聚合函数（如SUM、AVG、MAX等）默认会忽略NULL值**，仅对**非NULL的数值**进行计算。

举个简单例子：

若有数据 1，NULL，2，NULL，执行 SUM(column)时，会忽略两个NULL，只计算 1+2=3，结果为3而非 NULL。

步骤2：分析D选项的表达式逻辑

D选项的SQL为：

```sql
select sum(case when name like '张%' then 1 else null end) as zhang_first_name
from student_table;
```

- **CASE WHEN的作用**：对每行数据判断"是否是张姓"：
  - 若name like '张%'（是张姓），返回 1；
  - 否则返回 null。
- **SUM的计算逻辑**：
  对CASE WHEN输出的**每一行结果**求和。由于SUM会忽略NULL，只有CASE WHEN返回1的行（即张姓学生的行）会被计入求和，NULL的行会被跳过。
  假设表中有N个张姓学生，那么CASE WHEN会对这N行返回1，对剩余行返回NULL。SUM计算时，会忽略所有NULL，只把这N个1相加，结果为N（实际张姓学生的数量），**不会是NULL**。

总结

SUM函数的设计逻辑是"忽略NULL，仅累加非NULL值"。D选项中，CASE WHEN通过else null确保只有"张姓行"返回有效数值（1），其余行返回NULL；而SUM会自动跳过这些NULL，只对1求和，因此结果必然是**张姓学生的实际数量**（非NULL）。

DISTINCT 是 SQL 中用于**去除查询结果中完全重复的行**的关键字。其核心是**基于整行所有列的值进行唯一性判断。**

一、基本语法

```sql
SELECT DISTINCT column1, column2, ...
FROM table_name
WHERE condition;
```

- **位置**：必须紧跟在 `SELECT`关键字之后。
- **作用范围**：作用于其后列出的**所有列的组合**，而不仅仅是第一列。

二、关键特性和详细说明

1. 基于所有选定列进行去重

DISTINCT 比较的是查询结果中**每一行所有列的值组合**。只有所有列的值都完全相同的行才会被视作重复，只保留其中一行。

```sql
-- 假设表中有数据：(1, 'A'), (1, 'B'), (1, 'A')
SELECT DISTINCT id, name FROM table;
-- 返回：(1, 'A'), (1, 'B') 两行，因为第三行(1,'A')与第一行完全重复
```

2. 对NULL值的处理

在DISTINCT的逻辑中，**所有NULL值被视为彼此相等**。因此，多个全为NULL的行（或多列组合中NULL出现在相同位置）会被去重，只保留一个NULL。

```sql
-- 假设某列值为：NULL, NULL, 'A', 'B', NULL
SELECT DISTINCT column FROM table;
-- 返回：NULL, 'A', 'B' （三个NULL被合并为一个）
```

3. 在聚合函数中的特殊用法：COUNT(DISTINCT column)

`COUNT(DISTINCT column)`用于统计**某列中不重复值**的个数，这是一个非常常用的聚合模式。

```sql
-- 统计价格共有多少种不同的取值
SELECT COUNT(DISTINCT price) FROM product;
-- 注意：COUNT(DISTINCT column) 会忽略该列的NULL值（不将其计入数量）
```

4. **与GROUP BY的本质区别**

特性	DISTINCT	GROUP BY
主要目的	去除重复行，只关心结果的唯一性。	先分组，然后通常配合聚合函数（如SUM, AVG）对每个组进行计算。
能否使用聚合函数	在SELECT子句中**不能**直接与未聚合的其他列混用。 错误示例：`SELECT DISTINCT dept, SUM(salary) ...`	是使用聚合函数的**前提**。正确示例：`SELECT dept, SUM(salary) ... GROUP BY dept`
性能与逻辑	数据库通常在找出最终结果后，最后一步执行去重。	数据库需要先明确分组逻辑，然后进行聚合计算。
结果排序	不保证结果顺序。	分组后，每个组通常会返回一行，结果集可能更小。

简单来说：`DISTINCT`用于"**去重**"，`GROUP BY`用于"**分组后计算**"。虽然有时两者能达到相似的去重效果，但语义和用途不同。

三、重要注意事项

1. **性能影响**：DISTINCT 需要对结果集进行排序和比较，在数据量大时可能带来性能开销。如果不需要去除所有列的重复，而只是部分列，考虑是否可用GROUP BY或子查询优化。
2. **语法位置错误**：DISTINCT 必须紧跟在SELECT之后。`SELECT column1, DISTINCT column2`是**错误语法**。
3. **与ALL对应**：`SELECT`默认等价于 `SELECT ALL`，即返回所有行（包括重复）。DISTINCT 是其反义操作。

在 SQL 中，`COUNT(DISTINCT column)`用于统计某列不重复值的个数，而 **`UNIQUE`并不是一个聚合函数**，因此不能像 `DISTINCT`那样直接在`COUNT()`中使用。两者在语义和用法上有本质区别。

一、核心区别

特性	`COUNT(DISTINCT column)`	`UNIQUE`（在 SQL 上下文中的常见含义）
性质	聚合函数的一部分，用于计算不重复值的数量。	通常是**约束**（如唯一约束）或**关键字**（在 SELECT 中作为 `DISTINCT`的同义词）。
用法	仅用于聚合函数内部，返回一个标量值（数字）。	1. 作为约束：`CREATE TABLE`时定义列或表的唯一性。 2. 在 SELECT 子句中，某些数据库支持 `SELECT UNIQUE ...`作为 `SELECT DISTINCT ...`的同义词。
示例	`SELECT COUNT(DISTINCT city) FROM users;`	约束：`CREATE TABLE t (id INT UNIQUE);` 查询：`SELECT UNIQUE city FROM users;`（部分数据库支持）

二、详细说明

1. COUNT(DISTINCT column)

- 这是 SQL **标准语法**，所有主流数据库（MySQL、PostgreSQL、SQL Server、Oracle 等）都支持。
- 它统计某列中**去除 NULL 和重复值后**的不同值的数量。

sql

```sql
-- 示例：统计 products 表中不同 price 的个数
SELECT COUNT(DISTINCT price) FROM products;
```

2. UNIQUE

- **作为约束**：用于确保列中的值唯一（类似主键但允许 NULL）。

sql

```sql
CREATE TABLE employees (
    email VARCHAR(100) UNIQUE  -- 确保 email 不重复
);
```

SQL主要分成四部分：

（1）数据定义。（SQL DDL）用于定义SQL模式、基本表、视图和索引的创建和撤消操作。
（2）数据操纵。（SQL DML）数据操纵分成数据查询和数据更新两类。数据更新又分成插入、删除、和修改三种操作。
（3）数据控制。（DCL）包括对基本表和视图的授权，完整性规则的描述，事务控制等内容。

（4）嵌入式SQL的使用规定。（TCL）涉及到SQL语句嵌入在宿主语言程序中使用的规则。

淘宝网站的 pv 和 uv：

PV（页面浏览量）

指用户每次打开或刷新一个页面就被记录一次。同一个用户浏览多个页面，PV会累计增加。它反映了网站或页面的总体被浏览次数。

UV（独立访客数）

指在一天内，访问网站的不同用户数量。同一用户无论当天访问多少次，都只计为1个UV。它反映了访问网站的“人数”规模。

答案为 A，考察的是 row_number，rank，dense_rank 3 个排名函数的用法。
row_number：不管排名是否有相同的，都按照顺序1，2，3 ..... n；
rank：排名相同的名次一样，同一排名有几个，后面排名就会跳过几次；
dense_rank：排名相同的名次一样，且后面名次不跳跃。

- 修改表：ALTER TABLE 表名 修改选项 。选项集合：
- { ADD COLUMN <列名> <类型> -- **增加列**
- | CHANGE [COLUMN] <旧列名> <新列名> <新列类型> -- **修改列名或类型**
- | ALTER [COLUMN] <列名> { SET DEFAULT <默认值> | DROP DEFAULT } -- **修改/删除 列的默认值**
- | MODIFY [COLUMN] <列名> <类型> -- **修改列类型**
- | DROP [COLUMN] <列名> -- **删除列**
- | RENAME TO <新表名> -- **修改表名**
- | CHARACTER SET <字符集名> -- **修改字符集**
- | COLLATE <校对规则名> } -- **修改校对规则（比较和排序时用到）**

一、数据定义语言 (DDL - Data Definition Language)

1. 数据库操作

sql

```sql
-- 创建数据库
CREATE DATABASE database_name
    CHARACTER SET utf8mb4
    COLLATE utf8mb4_unicode_ci;

-- 使用数据库
USE database_name;

-- 删除数据库
DROP DATABASE database_name;

-- 修改数据库字符集
ALTER DATABASE database_name
    CHARACTER SET utf8mb4;
```

2. 表操作

sql

```sql
-- 创建表
CREATE TABLE users (
    id INT UNSIGNED AUTO_INCREMENT PRIMARY KEY COMMENT '主键ID',
    username VARCHAR(50) NOT NULL UNIQUE COMMENT '用户名',
    email VARCHAR(100) COMMENT '邮箱',
    age TINYINT UNSIGNED DEFAULT 0 COMMENT '年龄',
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP COMMENT '创建时间',
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP
COMMENT '更新时间',
    status ENUM('active', 'inactive', 'banned') DEFAULT 'active' COMMENT '状态'
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COMMENT='用户表';

-- 修改表结构
ALTER TABLE users
    ADD COLUMN phone VARCHAR(20) AFTER email,
    MODIFY COLUMN age SMALLINT UNSIGNED,
    DROP COLUMN phone,
    CHANGE COLUMN username user_name VARCHAR(50);

-- 删除表
DROP TABLE IF EXISTS users;

-- 清空表（重置自增ID）
TRUNCATE TABLE users;

-- 重命名表
RENAME TABLE old_name TO new_name;
```

使用场景:

- CREATE TABLE ： 系统初始化、新功能模块开发
- ALTER TABLE ： 需求变更、字段扩展、性能优化（添加索引）
- TRUNCATE ： 日志清理、测试数据重置（比DELETE快，不可回滚）

3. 索引管理

```sql
-- 创建索引
CREATE INDEX idx_username ON users(username);
CREATE UNIQUE INDEX idx_email ON users(email);
CREATE FULLTEXT INDEX idx_content ON articles(content); -- 全文索引

-- 复合索引（最左前缀原则）
CREATE INDEX idx_name_age ON users(username, age);

-- 删除索引
DROP INDEX idx_username ON users;

-- 查看索引
SHOW INDEX FROM users;
```

二、数据操作语言 (DML - Data Manipulation Language)

1. 插入数据 (INSERT)

```sql
-- 单行插入
INSERT INTO users (username, email, age)
VALUES ('张三', 'zhangsan@example.com', 25);

-- 多行插入（推荐，减少网络往返）
INSERT INTO users (username, email, age)
VALUES
    ('李四', 'lisi@example.com', 30),
    ('王五', 'wangwu@example.com', 28),
    ('赵六', 'zhaoliu@example.com', 35);

-- 插入或更新（UPSERT）
INSERT INTO users (id, username, age)
VALUES (1, '张三', 26)
ON DUPLICATE KEY UPDATE
    age = VALUES(age),
    updated_at = NOW();
-- 插入忽略（忽略重复键错误）
INSERT IGNORE INTO users (username, email)
VALUES ('张三', 'new@example.com');

-- 从其他表插入
INSERT INTO users_backup SELECT * FROM users WHERE created_at < '2023-01-01';
```

使用场景：

- ON DUPLICATE KEY UPDATE ：数据同步、缓存更新、计数器（如点赞数）
- INSERT IGNORE ：批量导入时跳过重复数据
- 多行插入：大数据量导入优化

2. 查询数据 (SELECT)

```sql
-- 基础查询
SELECT * FROM users WHERE status = 'active';

-- 指定字段（减少IO，提高性能）
SELECT id, username, email FROM users;

-- 条件查询
SELECT * FROM users
```

2. **查询数据** (SELECT)

sql

```sql
-- 基础查询
SELECT * FROM users WHERE status = 'active';

-- 指定字段（减少IO，提高性能）
SELECT id, username, email FROM users;

-- 条件查询
SELECT * FROM users
WHERE age BETWEEN 18 AND 30
    AND status = 'active'
    AND email IS NOT NULL;

-- 模糊查询
SELECT * FROM users WHERE username LIKE '张%';  -- 以张开头的
SELECT * FROM users WHERE username LIKE '%三';  -- 以三结尾的
SELECT * FROM users WHERE username LIKE '%三%'; -- 包含三的（性能差，大数据量避免）
```

```sql
-- 排序
SELECT * FROM users
ORDER BY age DESC, created_at ASC
LIMIT 10 OFFSET 20;  -- 分页：每页10条，第3页

-- 聚合查询
SELECT
    status,
    COUNT(*) as total_count,
    AVG(age) as avg_age,
    MAX(age) as max_age,
    MIN(age) as min_age,
    SUM(score) as total_score
FROM users
WHERE created_at >= '2024-01-01'
GROUP BY status
HAVING COUNT(*) > 5;  -- 过滤聚合结果
```

```sql
-- 去重
SELECT DISTINCT department FROM employees;

-- 连接查询
-- 内连接（只返回匹配的行）
SELECT u.*, o.order_id
FROM users u
INNER JOIN orders o ON u.id = o.user_id;

-- 左连接（返回左表所有行，右表不匹配为NULL）
SELECT u.*, o.order_id
FROM users u
LEFT JOIN orders o ON u.id = o.user_id
WHERE o.order_id IS NULL;  -- 查找没有订单的用户

-- 右连接
SELECT * FROM users u
RIGHT JOIN departments d ON u.dept_id = d.id;
```

```sql
-- 全外连接（MySQL不支持，用UNION模拟）
SELECT * FROM users u
LEFT JOIN departments d ON u.dept_id = d.id
UNION
SELECT * FROM users u
RIGHT JOIN departments d ON u.dept_id = d.id;

-- 子查询
-- IN子查询
SELECT * FROM users WHERE id IN (SELECT user_id FROM orders WHERE amount > 1000);

-- EXISTS子查询（通常比IN高效）
SELECT * FROM users u WHERE EXISTS (
    SELECT 1 FROM orders o WHERE o.user_id = u.id AND o.amount > 1000
);

-- 关联子查询
SELECT * FROM users u1
WHERE age > (SELECT AVG(age) FROM users u2 WHERE u2.department = u1.department);
```

```sql
-- 联合查询
SELECT name, email FROM customers
UNION ALL  -- ALL保留重复，UNION去重
SELECT name, email FROM suppliers;

-- 窗口函数（MySQL 8.0+）
SELECT
    id,
    username,
    age,
    ROW_NUMBER() OVER (ORDER BY age DESC) as rank,
    RANK() OVER (ORDER BY age DESC) as rank_with_gap,
    DENSE_RANK() OVER (ORDER BY age DESC) as dense_rank,
    NTILE(4) OVER (ORDER BY age) as quartile,
    LAG(age, 1) OVER (ORDER BY created_at) as prev_age,
    LEAD(age, 1) OVER (ORDER BY created_at) as next_age,
    AVG(age) OVER (PARTITION BY department) as dept_avg_age
FROM users;
```

使用场景:

- EXISTS vs IN ：外表小用IN，内表小用EXISTS
- LEFT JOIN + IS NULL ：查找缺失关联（如未下单用户）
- 窗口函数：排行榜、同比环比计算、分组TopN

3. 更新数据 (UPDATE)

sql

```sql
-- 基础更新
UPDATE users SET age = 26, updated_at = NOW() WHERE id = 1;

-- 批量更新（务必加WHERE条件）
UPDATE users SET status = 'inactive' WHERE last_login < '2023-01-01';

-- 关联更新
UPDATE users u
JOIN orders o ON u.id = o.user_id
SET u.total_spent = u.total_spent + o.amount
WHERE o.status = 'completed' AND o.id = 123;

-- 使用子查询更新
UPDATE users SET level = 'vip'
WHERE id IN (SELECT user_id FROM orders GROUP BY user_id HAVING SUM(amount) >
10000);

-- 限制更新数量
UPDATE users SET status = 'processed'
WHERE status = 'pending'
LIMIT 100;
```

4. 删除数据 (DELETE)

```sql
-- 删除指定行
DELETE FROM users WHERE id = 1;

-- 批量删除（分批删除避免锁表）
DELETE FROM users WHERE created_at < '2022-01-01' LIMIT 1000;

-- 关联删除
DELETE u FROM users u
LEFT JOIN orders o ON u.id = o.user_id
WHERE o.id IS NULL AND u.created_at < '2023-01-01';

-- 清空表（快速，不可回滚，重置自增ID）
TRUNCATE TABLE users;
```

三、数据控制语言 (DCL - Data Control Language)

```sql
-- 创建用户
CREATE USER 'app_user'@'%' IDENTIFIED BY 'strong_password';

-- 授权
GRANT SELECT, INSERT, UPDATE ON database_name.* TO 'app_user'@'%';
GRANT ALL PRIVILEGES ON *.* TO 'admin'@'localhost';

-- 撤销权限
REVOKE DELETE ON database_name.* FROM 'app_user'@'%';

-- 刷新权限
FLUSH PRIVILEGES;

-- 查看权限
SHOW GRANTS FOR 'app_user'@'%';
```

四、事务控制语言 (TCL - Transaction Control Language)

```sql
-- 开启事务
START TRANSACTION;
-- 或 BEGIN;

-- 保存点
SAVEPOINT before_update;

-- 提交事务
COMMIT;

-- 回滚事务
ROLLBACK;
ROLLBACK TO SAVEPOINT before_update;  -- 回滚到保存点

-- 自动提交控制
SET autocommit = 0;  -- 关闭自动提交
SET autocommit = 1;  -- 开启自动提交（默认）

-- 事务隔离级别设置
SET SESSION TRANSACTION ISOLATION LEVEL READ COMMITTED;
```

★ 五、常用函数详解

五、常用函数详解

1. 字符串函数

表格

函数	说明	使用场景
CONCAT(str1, str2, ...)	字符串拼接	生成完整地址、姓名拼接
CONCAT_WS(sep, str1, ...)	用分隔符拼接	CSV生成、路径拼接
SUBSTRING(str, pos, len)	截取子串	身份证号脱敏、手机号隐藏
LEFT(str, len) / RIGHT(str, len)	左/右截取	提取区号、后缀判断
LENGTH(str)	字节长度	存储空间计算
CHAR_LENGTH(str)	字符长度	中文长度统计
TRIM(str)	去除两端空格	数据清洗
REPLACE(str, from, to)	替换字符串	敏感词过滤、格式统一
UPPER(str) / LOWER(str)	大小写转换	验证码不区分大小写
INSTR(str, substr)	查找子串位置	判断包含关系
LPAD(str, len, pad) / RPAD	左/右填充	订单号固定位数、对齐显示
FORMAT(num, decimals)	格式化数字	金额千分位显示

2. 数值函数

表格

函数	说明	使用场景
ABS(x)	绝对值	计算偏差
ROUND(x, d)	四舍五入	金额计算
CEIL(x) / FLOOR(x)	向上/下取整	分页计算、库存箱数
MOD(n, m)	取模	奇偶判断、分组轮询
RAND()	随机数	随机排序、抽奖
POW(x, y) / SQRT(x)	幂运算/开方	距离计算、评分算法
GREATEST(v1, v2, ...)	最大值	多字段比较
LEAST(v1, v2, ...)	最小值	多字段比较

3. 日期时间函数

表格

函数	说明	使用场景
NOW() / CURRENT_TIMESTAMP	当前日期时间	记录创建时间
CURDATE() / CURRENT_DATE	当前日期	按天统计
CURTIME() / CURRENT_TIME	当前时间	时段分析
DATE(expr)	提取日期部分	去除时间部分
TIME(expr)	提取时间部分	时段提取
YEAR() / MONTH() / DAY()	提取年月日	分组统计
HOUR() / MINUTE() / SECOND()	提取时分秒	时段分析
WEEK() / WEEKDAY() / DAYOFWEEK()	周信息	周报统计
DATE_ADD(date, INTERVAL expr unit)	日期加减	计算到期日、有效期
DATE_SUB(date, INTERVAL expr unit)	日期减法	历史数据查询
DATEDIFF(date1, date2)	天数差	计算天数间隔
TIMESTAMPDIFF(unit, d1, d2)	指定单位差	精确年龄计算
DATE_FORMAT(date, format)	格式化日期	报表展示
STR_TO_DATE(str, format)	字符串转日期	数据导入
UNIX_TIMESTAMP() / FROM_UNIXTIME()	时间戳转换	与程序交互
LAST_DAY(date)	当月最后一天	月结计算

4. 聚合函数

表格

函数	说明	使用场景
COUNT(*)	计数所有行	统计总量
COUNT(expr)	计数非NULL	统计有效数据
COUNT(DISTINCT expr)	去重计数	统计UV、去重用户
SUM(expr)	求和	销售额统计
AVG(expr)	平均值	平均分、均价
MAX(expr) / MIN(expr)	最大/最小值	峰值、极值
GROUP_CONCAT(expr)	分组拼接	多对一展示

5. **条件判断函数**

函数	说明	使用场景
IF(expr, v1, v2)	条件判断	简单二选一
IFNULL(expr, v2)	NULL处理	默认值设置
NULLIF(expr1, expr2)	相等返回NULL	避免除零错误
CASE WHEN ... THEN ... END	多条件判断	复杂逻辑、数据转换

4. `CASE WHEN ... THEN ... END`

- **说明**：标准的 SQL 条件表达式，支持多条件分支。有两种形式：
  - **简单 CASE**：`CASE 列 WHEN 值1 THEN 结果1 WHEN 值2 THEN 结果2 ELSE 默认值 END`
  - **搜索 CASE**：`CASE WHEN 条件1 THEN 结果1 WHEN 条件2 THEN 结果2 ELSE 默认值 END`
- **使用场景**：复杂的多条件判断、数据转换、自定义分组等。

6. **类型转换函数**

函数	说明	使用场景
CAST(expr AS type)	类型转换	数据类型统一
CONVERT(expr, type)	类型转换	字符集转换
BINARY(expr)	转二进制	区分大小写比较

7. **JSON函数（MySQL 5.7+）**

函数	说明	使用场景
JSON_OBJECT(key, val, ...)	创建JSON对象	动态属性存储
JSON_ARRAY(val1, val2, ...)	创建JSON数组	列表存储
JSON_EXTRACT(json, path)	提取JSON值	查询JSON字段
JSON_SET(json, path, val)	设置JSON值	更新JSON
JSON_REMOVE(json, path)	删除JSON值	清理字段
JSON_CONTAINS(json, val)	判断是否包含	条件查询
JSON_KEYS(json)	获取所有键	结构分析
JSON_UNQUOTE(json)	去引号	提取纯文本

8. **窗口函数（MySQL 8.0+）**

表格

函数	说明	使用场景
ROW_NUMBER()	行号（唯一）	精确排名
RANK()	排名（跳号）	竞赛排名
DENSE_RANK()	密集排名（不跳号）	等级划分
NTILE(n)	分桶	百分位、四分位
LAG(expr, n)	前N行值	环比计算
LEAD(expr, n)	后N行值	同比计算
FIRST_VALUE(expr)	分区首值	基准比较
LAST_VALUE(expr)	分区尾值	最新值
SUM/AVG/COUNT(...) OVER(...)	滑动聚合	移动平均

方法一：需要单独命名avg_question

select university,avg(question_cnt) AS avg_question_cnt from user_profile

group by university

order by avg_question_cnt

方法二：不需要单独命名

select university,avg(question_cnt) from user_profile

group by university

order by avg(question_cnt)

这样操作能运行成功的原因是 order by后面可以加聚合函数。但要注意，group by后面不可

有三类后面可以加聚合函数

1、select

2、order by

3、having

不一定。表连接在 SQL 中主要有两种写法：

1. **显式连接**：在 `FROM` 子句中使用 `JOIN` 关键字（如 `INNER JOIN`、`LEFT JOIN` 等）并配合 `ON` 指定连接条件。
2. **隐式连接**：在 `FROM` 子句中用逗号分隔多个表，然后在 `WHERE` 子句中写连接条件。

虽然两种方式都能实现表连接，但现代 SQL 强烈推荐使用**显式连接**语法（即 `FROM ... JOIN ... ON`），因为它更清晰、易读、易于维护，并且能避免意外的笛卡尔积。

`DISTINCT` 是 SQL 中用于**去除查询结果中重复行**的关键字。当查询可能返回多条重复记录时，使用 `DISTINCT` 可以确保结果集中的每一行都是唯一的。

---

1. 基本语法

sql 复制 下载

```sql
SELECT DISTINCT 列1, 列2, ...
FROM 表名
[WHERE 条件];
```

- `DISTINCT` 放在 `SELECT` 后面，紧跟要查询的列。
- 它会对**所有选定列的组合**进行去重，即只有当多行在所有选定列上的值都相同时，才被视为重复行，只保留一行。

示例 1：查询所有不同的学校

sql

```sql
SELECT DISTINCT university
FROM user_profile;
```

返回 `user_profile` 表中所有不重复的学校名称。

示例 2：查询所有不同的 (学校, 性别) 组合

sql

```sql
SELECT DISTINCT university, gender
FROM user_profile;
```

返回学校与性别的所有唯一组合，例如（北京大学, 男）、（北京大学, 女）等。

2. 与聚合函数结合：`COUNT(DISTINCT ...)`

`DISTINCT` 经常与 `COUNT` 一起使用，统计某列中不同值的个数。

sql

```sql
SELECT COUNT(DISTINCT 列名) FROM 表名;
```

您提到的 `SELECT col1, DISTINCT col2` 这种写法在标准 SQL 中是**错误的。** `DISTINCT` 必须紧跟在 `SELECT` 关键字之后，作用于**后面所有列的组合**，不能单独作用于某一列。

Case when 语句：

方法一：CASE WHEN（推荐）

```sql
SELECT
    device_id,
    gender,
    CASE
        WHEN age < 20 THEN '20岁以下'
        WHEN age < 25 THEN '20-24岁'
        WHEN age >= 25 THEN '25岁及以上'
        ELSE '其它'
    END AS age_cut
FROM user_profile;
```

说明

- `CASE` 从第一个 `WHEN` 开始判断，一旦条件成立就返回对应的 `THEN` 值，并退出。
- `ELSE` 是可选的，用于处理所有条件都不满足的情况（如 `age` 为 `NULL`）。
- `END AS age_cut` 给计算列起别名，使结果集中出现 `age_cut` 列。

方法二：使用 IF() 函数（MySQL 特有）

```sql
SELECT
    device_id,
    gender,
    IF(age < 20, '20岁以下',
        IF(age < 25, '20-24岁',
            IF(age >= 25, '25岁及以上', '其它')
        )
    ) AS age_cut
FROM user_profile;
```

- `IF(condition, true_value, false_value)` 是函数，可以嵌套使用。
- 但嵌套过多时可读性较差，不如 `CASE` 清晰。

```sql
--法一：like运算符
select
    day(date) as day,
    count(question_id) as question_cnt
from question_practice_detail
where date like '2021-08%'
group by day(date);

--法二：regexp运算符
select
    day(date) as day,
    count(question_id) as question_cnt
from question_practice_detail
where date regexp '2021-08'
group by day(date);

--法三：substring提取日期
select
    day(date) as day,
    count(question_id) as question_cnt
from question_practice_detail
where substring(date,1,7) = '2021-08'
group by day(date);
```

1. 基本语法

sql 复制 下载

```sql
COUNT(DISTINCT 列1, 列2, ...)
```

- **作用**：统计指定列的组合值中，**不重复且非 NULL** 的行数。
- 它会对 `(列1, 列2, ...)` 形成的元组进行去重，只有当所有列的值都相同时才视为重复行，只计数一次。
- 如果组合中的**任一列为 NULL**，则该行不参与计数（因为 NULL 被认为是未知，不参与去重统计）。

下面详细介绍 SQL 中常用的四个文本处理函数：`REPLACE`、`SUBSTR`、`SUBSTRING_INDEX` 和 `TRIM`。每个函数都附有语法说明和具体示例，方便你理解其用法。

---

1. REPLACE 函数

作用：将字符串中的指定子串全部替换为另一个子串。

语法：

sql 复制 下载

```sql
REPLACE(str, from_str, to_str)
```

- `str`：原始字符串。
- `from_str`：要被替换的子串。
- `to_str`：替换后的新子串。

示例：

sql 复制 下载

```sql
SELECT REPLACE('hello world', 'world', 'MySQL') AS new_string;
-- 结果: 'hello MySQL'

SELECT REPLACE('apple apple pie', 'apple', 'orange') AS new_string;
-- 结果: 'orange orange pie'

-- 实际应用: 清洗电话字段中的分隔符
UPDATE users SET phone = REPLACE(phone, '-', '');
```

2. SUBSTR 函数

作用：从字符串中提取子串（MySQL 中也支持 `SUBSTRING`，两者等价）。

语法：

```sql
SUBSTR(str, pos, len)
```

- `str`：原始字符串。
- `pos`：开始提取的位置（从 1 开始计数）。若 `pos` 为负数，则从字符串尾部倒数。
- `len`：可选，提取的长度。若不指定，则提取从 `pos` 开始到字符串末尾的所有字符。

示例：

```sql
SELECT SUBSTR('abcdefg', 3, 2) AS sub;
-- 结果: 'cd'  (从第 3 个字符开始取 2 个字符)

SELECT SUBSTR('abcdefg', 3) AS sub;
-- 结果: 'cdefg'  (从第 3 个字符取到末尾)

SELECT SUBSTR('abcdefg', -3, 2) AS sub;
-- 结果: 'ef'  (从倒数第 3 个字符开始取 2 个, 即 'e','f')

-- 实际应用: 提取邮箱的用户名部分
SELECT SUBSTR(email, 1, LOCATE('@', email) - 1) AS username FROM users;
```

3. SUBSTRING_INDEX 函数

作用：按分隔符截取字符串，返回指定次数的分隔符之前（或之后）的子串。

语法：

sql 复制 下载

```sql
SUBSTRING_INDEX(str, delim, count)
```

- `str`：原始字符串。
- `delim`：分隔符。
- `count`：计数。
  - 如果 `count` 为正数，返回从左到右第 `count` 个分隔符**左边**的所有内容（包括左边第一个？实际上是返回前 count 个分隔符左侧的子串，即截取到第 count 次出现分隔符的位置，但不包括该分隔符）。
  - 如果 `count` 为负数，返回从右到左第 `|count|` 个分隔符**右边**的所有内容。

示例：

```sql
SELECT SUBSTRING_INDEX('www.mysql.com', '.', 2) AS part;
-- 结果: 'www.mysql' （返回前 2 个点左边的部分）

SELECT SUBSTRING_INDEX('www.mysql.com', '.', -2) AS part;
-- 结果: 'mysql.com' （返回最后 2 个点右边的部分）

-- 嵌套使用：提取中间部分（如提取域名的主机名）
SELECT SUBSTRING_INDEX(SUBSTRING_INDEX('www.mysql.com', '.', 2), '.', -1) AS middle;
-- 结果: 'mysql' （先取前 2 段得到 'www.mysql'，再取最后一段得到 'mysql'）

-- 实际应用：从文件路径中提取文件名
SELECT SUBSTRING_INDEX('/home/user/data.txt', '/', -1) AS filename;
-- 结果: 'data.txt'
```

4. TRIM 函数

作用：去除字符串首尾的空格或指定字符。

语法：

```sql
TRIM([{BOTH | LEADING | TRAILING} [remstr] FROM] str)
```

- `str`：要处理的字符串。
- `remstr`：可选，指定要去除的字符（默认为空格）。
- `BOTH`：去除首尾的指定字符（默认）。
- `LEADING`：仅去除开头的指定字符。
- `TRAILING`：仅去除结尾的指定字符。

示例：

```sql
SELECT TRIM('  hello  ') AS trimmed;
-- 结果: 'hello' （去除首尾空格）

SELECT TRIM(LEADING 'x' FROM 'xxxhelloxxx') AS trimmed;
-- 结果: 'helloxxx' （只去除开头的 'x'）

SELECT TRIM(TRAILING 'x' FROM 'xxxhelloxxx') AS trimmed;
-- 结果: 'xxxhello' （只去除结尾的 'x'）

SELECT TRIM(BOTH 'x' FROM 'xxxhelloxxx') AS trimmed;
-- 结果: 'hello' （去除首尾的 'x'）

-- 实际应用：清理用户输入的多余空格
UPDATE users SET name = TRIM(name);
```

★ 子查询末尾不能加分号，分号只能用于整个sql语句的最后！
聚合函数 min(index) 等要重新指定别名，不然无法引用！

您的 SQL 代码存在两处关键错误，导致执行失败：

1. **子查询中出现了多余的分号 `;`**
   在子查询内部，`order by university ;` 末尾的分号是不允许的，它会提前结束子查询的定义，导致后续的 `) as a` 等语法无法解析。分号只能用于结束整个 SQL 语句。
2. **未给聚合函数 `min(gpa)` 指定别名**
   子查询中 `SELECT university, min(gpa)` 返回的列默认名为 `min(gpa)`（包含括号）。在外部查询中，您试图用 `a.min(gpa)` 引用它，但列名含有特殊字符且未加反引号，并且这种写法本身就不合法。正确的做法是给聚合列起一个别名（如 `min_gpa`），然后在外部直接使用该别名。

此外，您使用了旧式的逗号连接（隐式连接），虽然能工作，但建议改用显式的 `JOIN` 语法，更清晰易懂。

★ ',' 逗号连接是隐式连接，只返回两个表中都存在的记录！

★ where语句是不能使用聚合函数（如sum），
但是是可以使用其它函数的（如month）！

1. WHERE 子句中使用函数是合法的

在 SQL 中，WHERE 子句用于过滤行，其条件表达式可以包含各种函数，包括：

- 日期函数：`YEAR()`、`MONTH()`、`DAY()`、`DATE_ADD()` 等
- 字符串函数：`UPPER()`、`LOWER()`、`SUBSTRING()`、`CONCAT()` 等
- 数学函数：`ABS()`、`ROUND()`、`FLOOR()` 等
- 聚合函数除外（如 `SUM`、`COUNT` 不能在 WHERE 中使用，因为它们需要分组后才能计算，但 `HAVING` 可以）

所以，`WHERE MONTH(date) = 8 AND YEAR(date) = 2021` 是完全正确的，它提取 `date` 列的月份和年份，然后判断是否等于 8 和 2021。

这个正则表达式用于匹配符合题目要求的两种电话号码格式：

```text
^([1-9][0-9]{9}|[1-9][0-9]{2}-[0-9]{3}-[0-9]{4})$
```

1. 整体结构

- `^`：匹配字符串的开始位置，确保整个字符串从开头就被检查。
- `$`：匹配字符串的结束位置，确保字符串结尾之后没有多余字符。
- `(...|...)`：括号内是一个分组，使用 `|` 表示“或”的关系，即整个表达式匹配两种模式中的任意一种。

---

2. 第一种模式：连续10位数字（第一位非0）

`[1-9][0-9]{9}`

- `[1-9]`：匹配第一位数字，范围是 `1` 到 `9`，保证了不以 `0` 开头。
- `[0-9]{9}`：匹配后续的 9 位数字（`0` 到 `9` 均可）。
- 总共 `1 + 9 = 10` 位数字。

匹配示例：`1234567890`、`9876543210`

---

3. 第二种模式：带短横的分段格式（第一位非0）

`[1-9][0-9]{2}-[0-9]{3}-[0-9]{4}`

- `[1-9]`：第一位数字不能是 `0`。
- `[0-9]{2}`：接着两位数字（可以是 `0` 到 `9`）。
  - 至此，第一部分 `xxx` 共三位，且首位非0。
- `-`：字面短横分隔符。
- `[0-9]{3}`：中间三位数字。
- `-`：第二个短横分隔符。
- `[0-9]{4}`：最后四位数字。
- 总字符数为：3位数字 + 1个短横 + 3位数字 + 1个短横 + 4位数字 = 12个字符，但数字部分共 3+3+4=10 位数字。

匹配示例：`123-456-7890`、`987-654-3210`

4. 为什么不匹配的例子

- `0123456789`：第一位是 `0`，不满足 `[1-9]` 要求。
- `123-4567-890`：中间部分是4位数字，不符合 `[0-9]{3}`。
- `123456789`：只有9位数字，不满足 `{9}` 要求。
- `123-456-78901`：末尾多了一位数字，不满足 `$` 结束条件。

---

5. 在SQL中的使用

在 MySQL 中，`REGEXP` 操作符用于判断字符串是否匹配给定的正则表达式。该查询返回所有 `phone_number` 列匹配上述模式的记录，并按 `id` 升序排列。

sql 复制

```sql
SELECT * FROM contacts
WHERE phone_number REGEXP '^([1-9][0-9]{9}|[1-9][0-9]{2}-[0-9]{3}-[0-9]{4})$'
ORDER BY id;
```

解析：基本数学函数使用

abs(num) 表示 取数值的绝对值 -1 --> 1

ceiling(num) 表示 向上取整 3.14 ---> 4 (把小数点后面的去掉 加一)

floor(num) 表示 向下取整 3.14 ---> 3 （直接把小数点后面的去掉）

round(num,n) 表示 保留几位小数(四舍五入) round(3.14,1) = 3.1